Introduction à la Programmation et l'Algorithmique

CHRISTELLE CAILLOUET – CHRISTELLE.CAILLOUET@UNIV-COTEDAZUR.FR

DINO LOPEZ PACHECO – DINO.LOPEZ@UNIV-COTEDAZUR.FR

C Libraries

Code reusability

- We frequently program a main function along other pieces of code
  - Macros, constants
  - Functions
  - And we rewrite (or copy/paste) similar pieces of code
- Libraries
  - Solution to write a code once, and reuse it multiple times
  - Provide compiled code
- Link the functions provided by a library with our C program

Creating a Library Code

1. Create the interface
   - Create a .h file (e.g. yourlib.h)
2. Implement the code
   - Create a .c file (e.g. yourlib.c)
3. Compile the library
   - Generate a .o file (using the -c option of gcc: gcc -o yourlib.o -c yourlib.c)
     - The *object code*
   - Eventually,
     - create a dynamically linked shared object file (.so file) from potentially multiples .o files
     - create a statically linked archive file (.a file) from potentially multiple .o files
4. Include your library where you need it (`#include "yourlib.h"`)
   - Compile & Link your new C program with your object code: gcc myprog.c yourlib.o

Create the interface: myarray.h

- Add include guards
  - Usually based on the file name
  - Avoids including a single library multiple times
- Add minimum number of headers
  - Headers needed to compile the header
- Add only declarations
  - Functions, structures, variables
- Global variable must be declared with the extern qualifier
- Avoid declaring internal functions that are not part of the library

```
#ifndef MYARRAY_H
#define MYARRAY_H
typedef struct intArray {
    int capacity;
    int size;
    int *array;
    int *next;
} MyArray;
extern int myVar; // initializing myVar here leads to errors
MyArray *createArray(int n);
void insert2dynArray(MyArray *ma, int val);
void deleteFromArray(MyArray *ma, int index);
void showArray(MyArray *ma, int showElems);
void deleteArray(MyArray *ma);
#endif
```

Implementing the code: myarray.c

- #include your interface first, then any other .h file
  - Include only needed .h files
- Implement your functions
  - Some functions might not be listed in the interface .h file (internal functions)
- there is no a main() function
- Compile your library
  - Generate the object code myarray.o with "gcc -c myarray.c"

```
#include "myarray.h"
#include <stdio.h>
#include <stdlib.h>

int myVar = 10; // right place to initialize
myVar

MyArray *createArray(int n) {…}
void insert2array(MyArray *ma, int val) {…}
int addSpace(MyArray *ma) {…}
void insert2dynArray(MyArray *ma, int val) {…}
void deleteFromArray(MyArray *ma, int index) {…}
void showArray(MyArray *ma, int showElems) {…}
void deleteArray(MyArray *ma) {…}
```

Including your library

```
.
|--- mylib --- myarray.o
|       |------ myarray.h
|       |------ myarray.c
|
|--- testlib.c
```

- #include your interface file (.h)
  - do not include the .c file
- Compile & Lin your testlib.c file
  - gcc -o testlib testlib.c mylib/myarray.o

```
#include <stdio.h>
#include "mylib/myarray.h"


int main() {
    printf("Testing our new C
library\n");
    MyArray *ma = createArray(0);
    insert2dynArray(ma, 10);
    showArray(ma, 1);
    deleteArray(ma);
}
```